

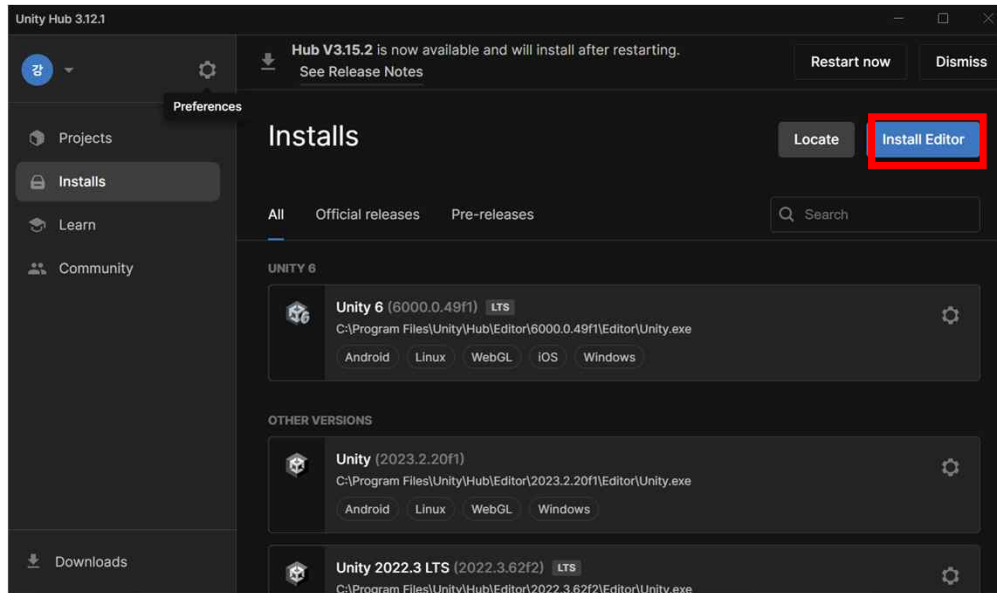
01

Unity 엔진 소개

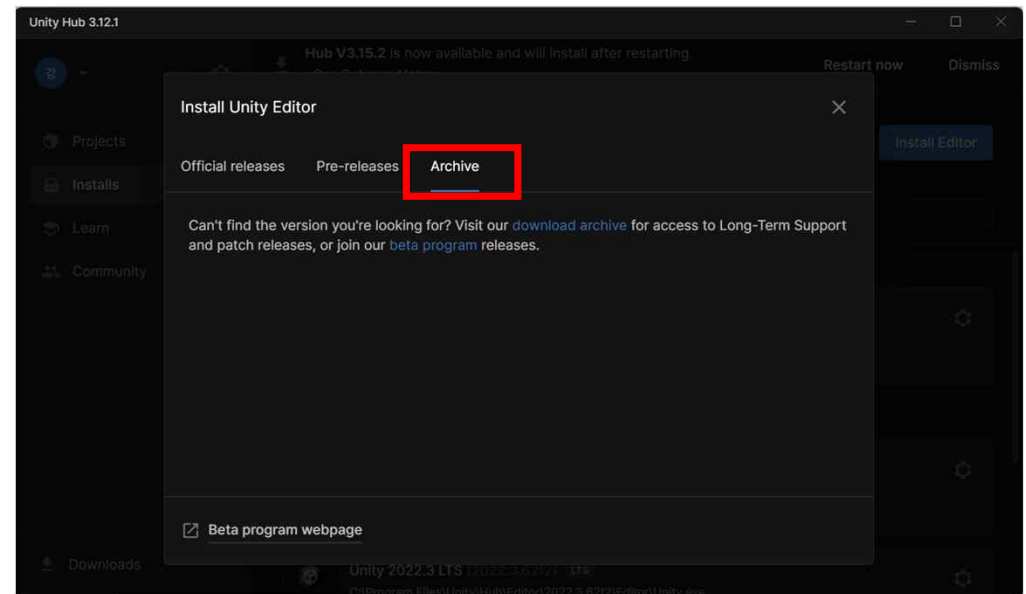


Unity 엔진 설치

1. 유니티 허브 설치 <https://unity.com/kr/download>
2. 유니티 허브 실행 및 로그인
3. **Install Editor** 버튼 클릭



4. **Archive** 클릭 ➡ **6000.049f1** 다운로드

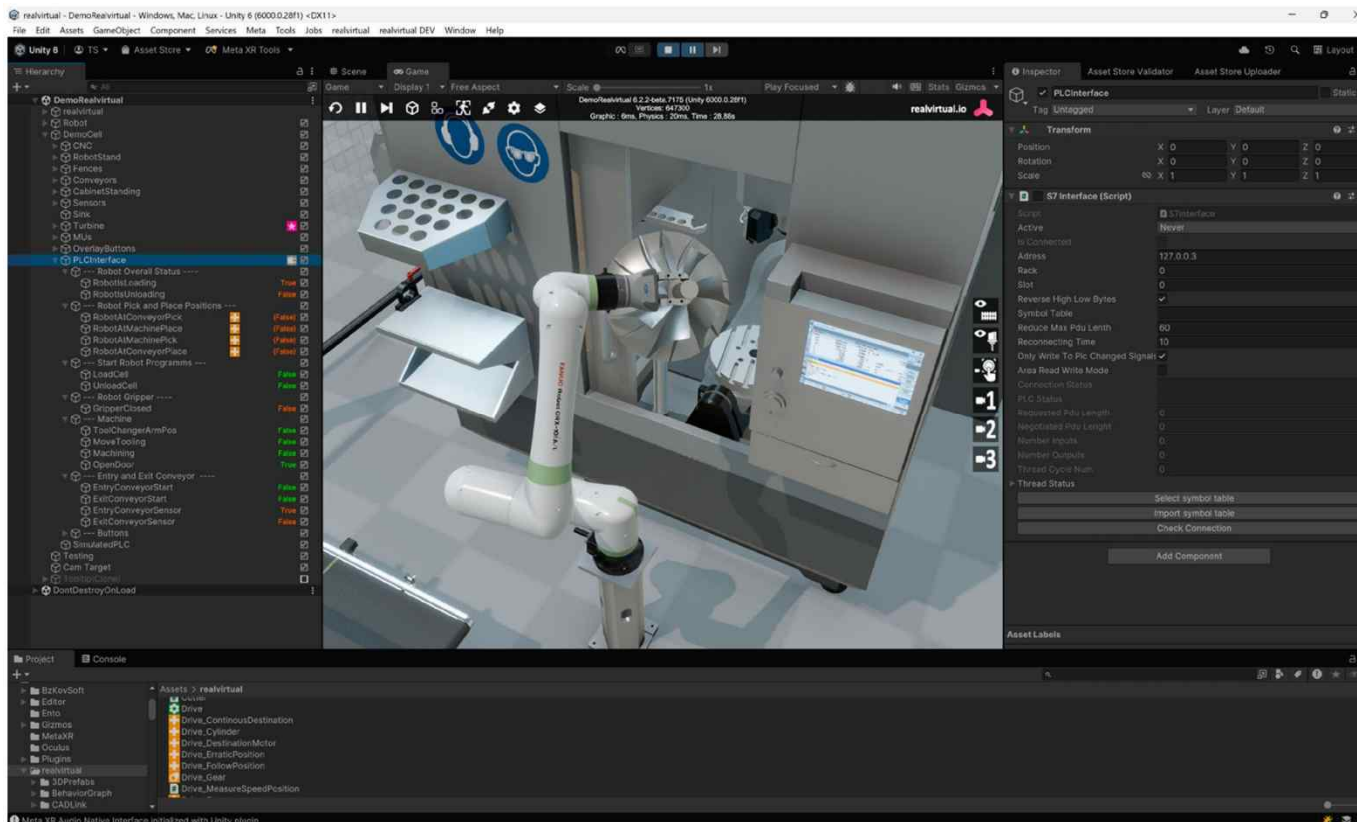


Unity 엔진

- 2D 및 모바일 게임 특화
- 애니메이션, 영화 등 다양한 콘텐츠 제작 툴
- Android, iOS, WebGL, XR 디바이스 등 다양한 플랫폼 지원



Unity 엔진



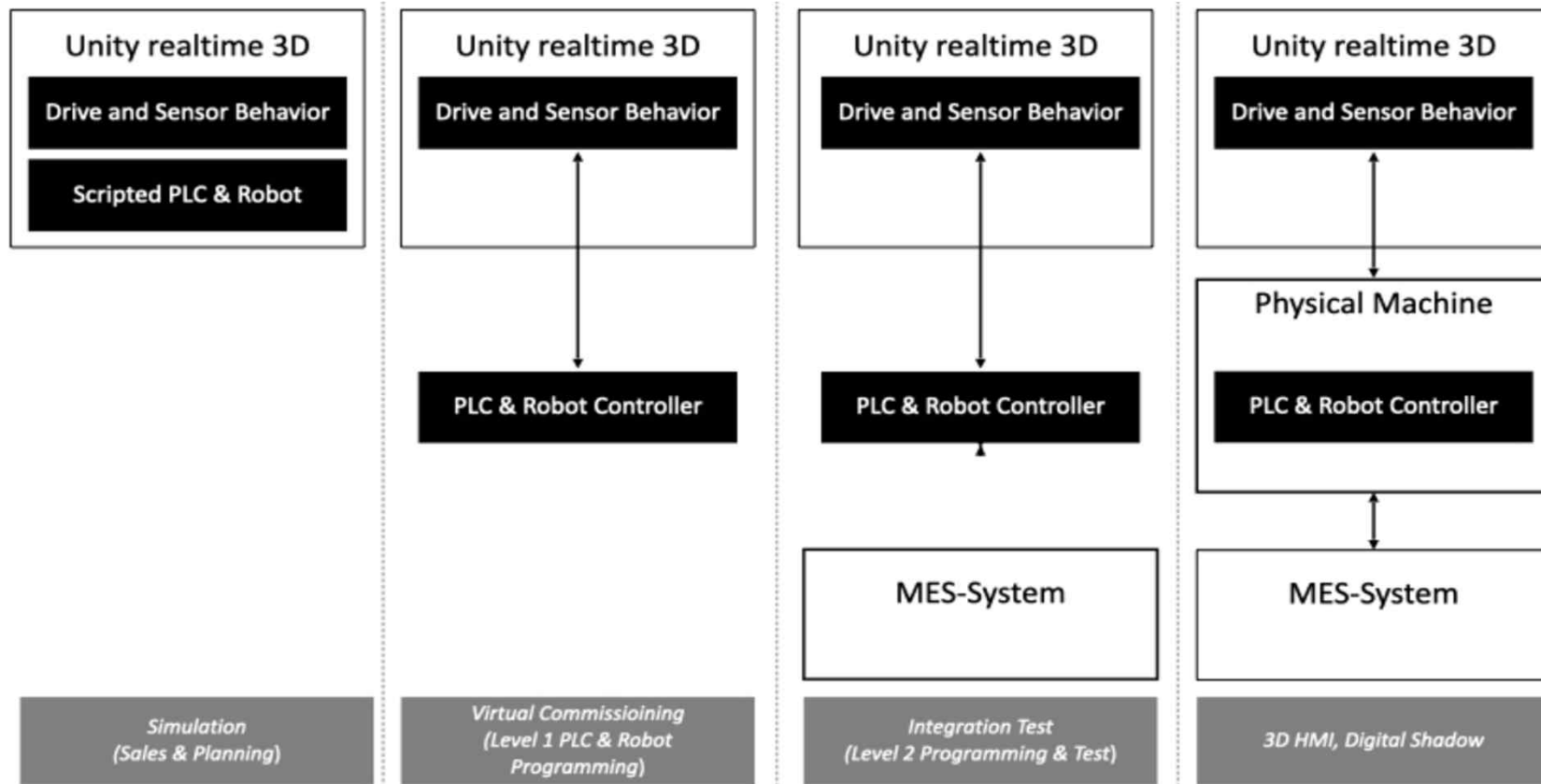
디지털 트윈 지원

- 스마트 CAD импорт
- 키네마틱 계층 정의
- 동작 시뮬레이션 설계
- 센서 및 피드백 루프 연동
- 씬의 모듈성 구성

디지털 트윈을 위한 AI

- ONNX 지원으로 AI 모델 직접 연동
- 합성 훈련 데이터세트 대규모 생성

Unity 엔진 활용 디지털 트윈의 4가지 레벨

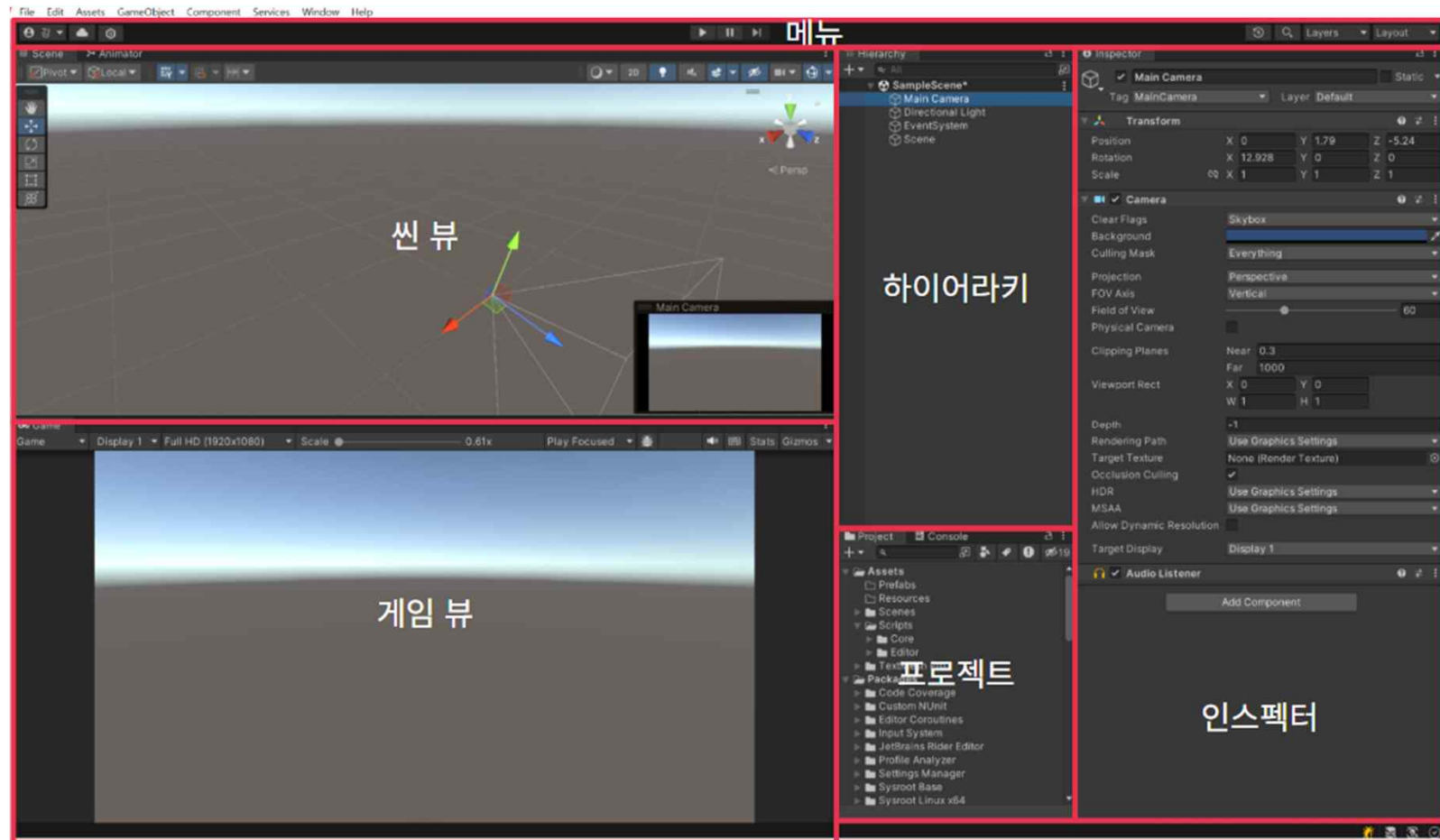


02

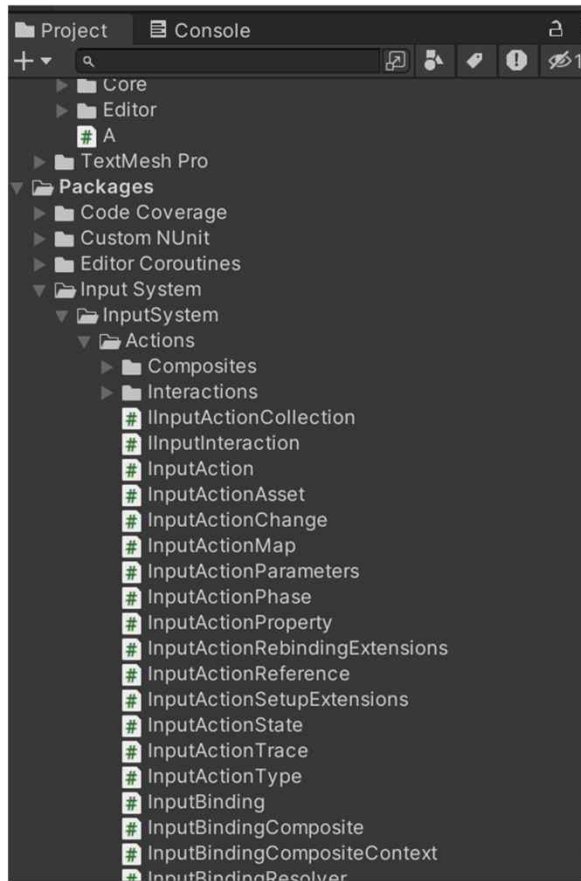
Unity 기초



Unity 인터페이스



Unity 엔진 구조

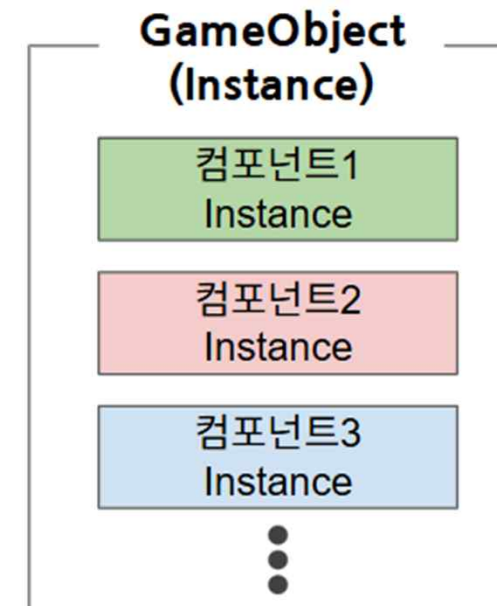
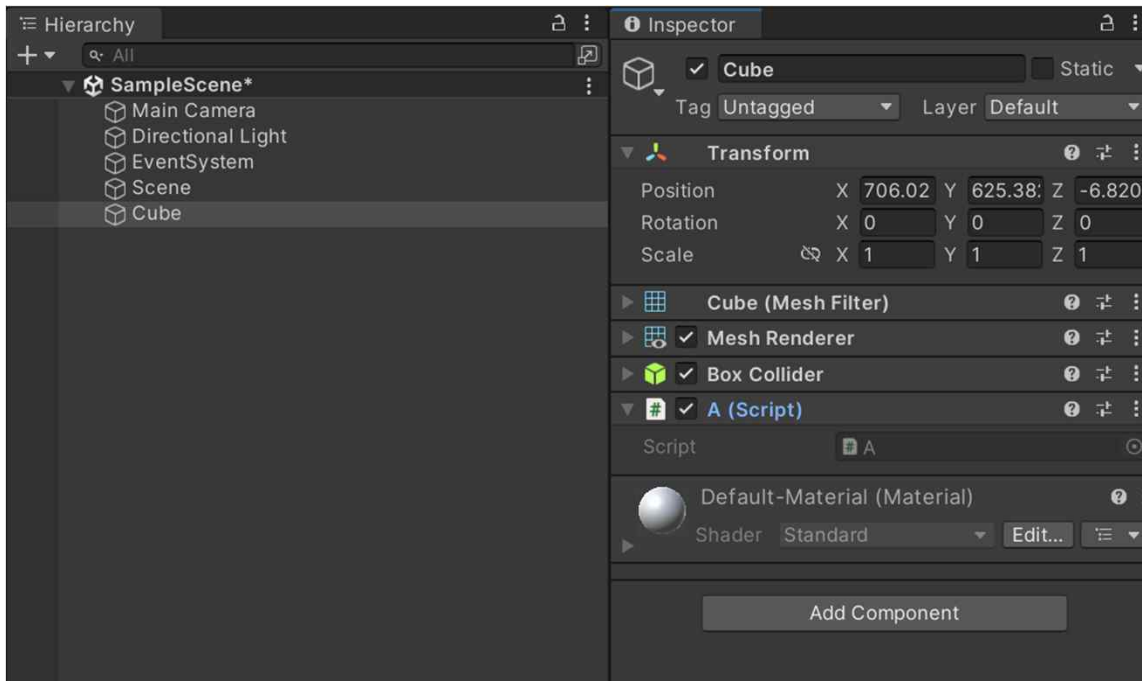


- 유니티는 클래스 기반 설계
- 엔진 코어는 C++로 작성, 감춰져 있음
- 회사에 따라 네이티브 엔진 코드 일부 제공

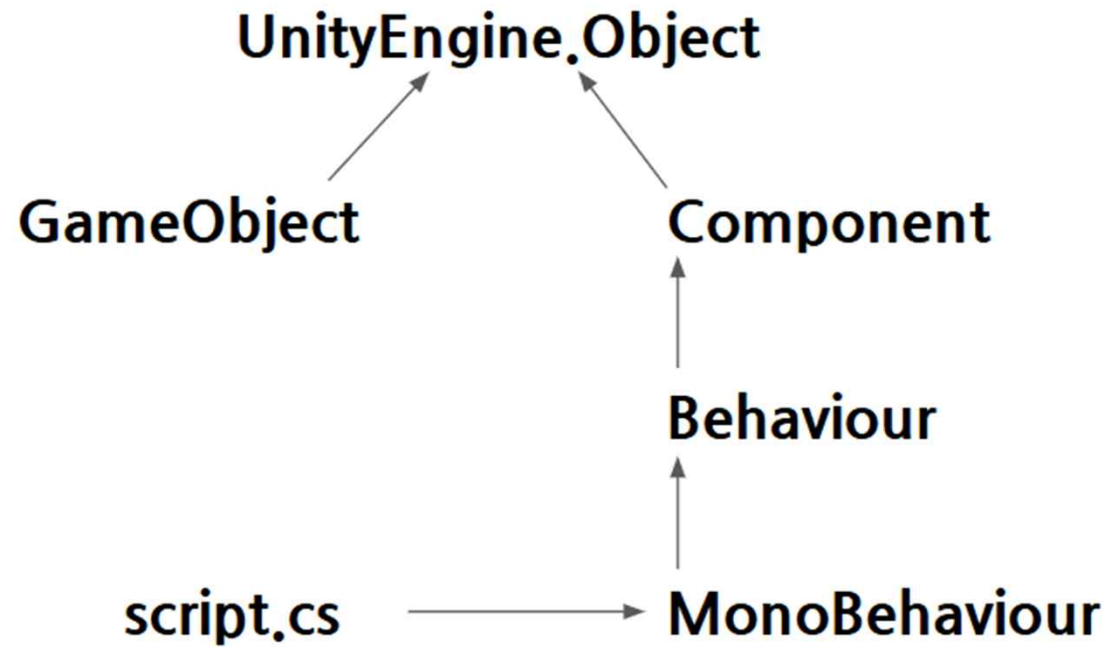
```
public sealed class InputAction : ICloneable, IDisposeable
{
    /// <summary>
    /// Name of the action.
    /// </summary>
    /// <value>Plain-text name of the action.</value>
    /// <remarks>
    /// Can be null for anonymous actions created in
    ///
    /// If the action is part of an <see cref="InputActionMap">
    /// will be unique in the map. The name is just
```


Unity - GameObject

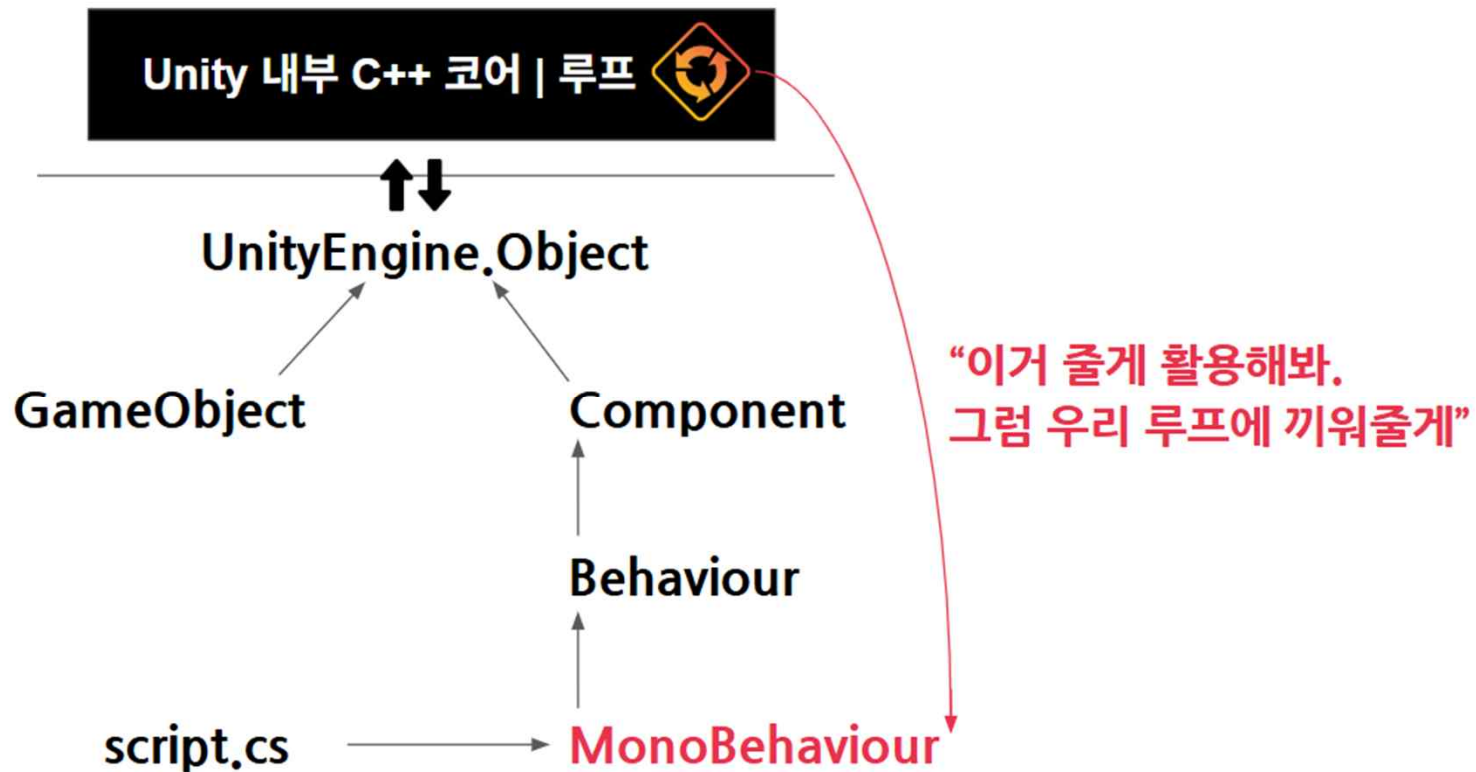
- GameObject(의 인스턴스)는 유니티 씬에서 모든 개체의 기본이 되는 단위
- 껍데기 기능만 하고 실제적인 기능을 하는 **컴포넌트(Component) 인스턴스들의 컨테이너 역할** 수행



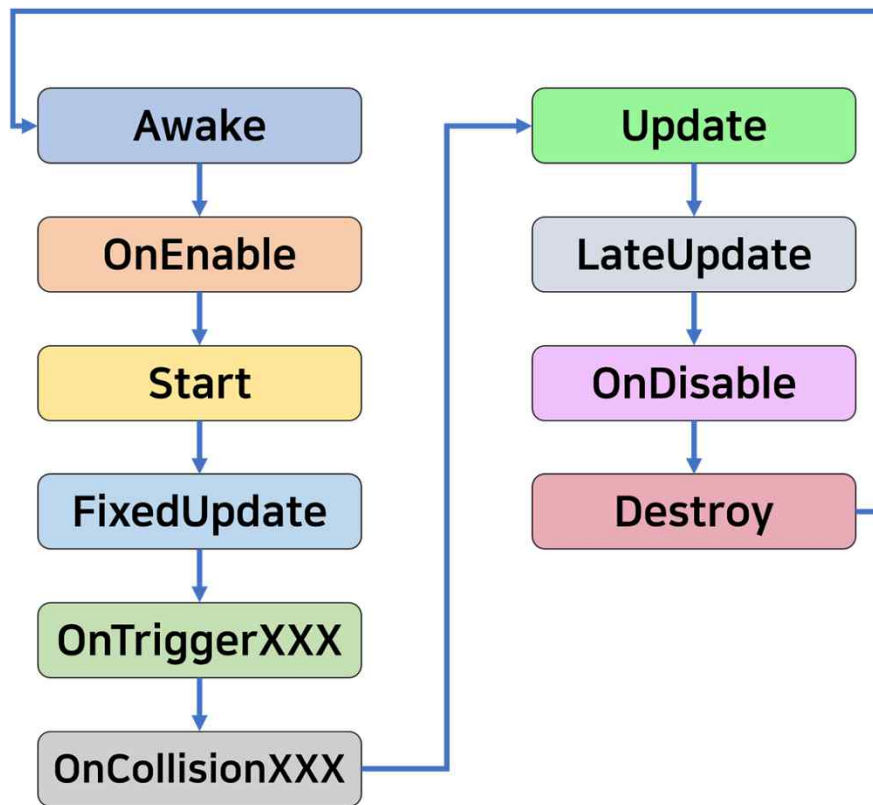
Unity - 상속 구조



Unity – MonoBehaviour



Unity – 생명주기 함수



스크립트(컴포넌트)가 실행되는 정해진 순서(SOP)

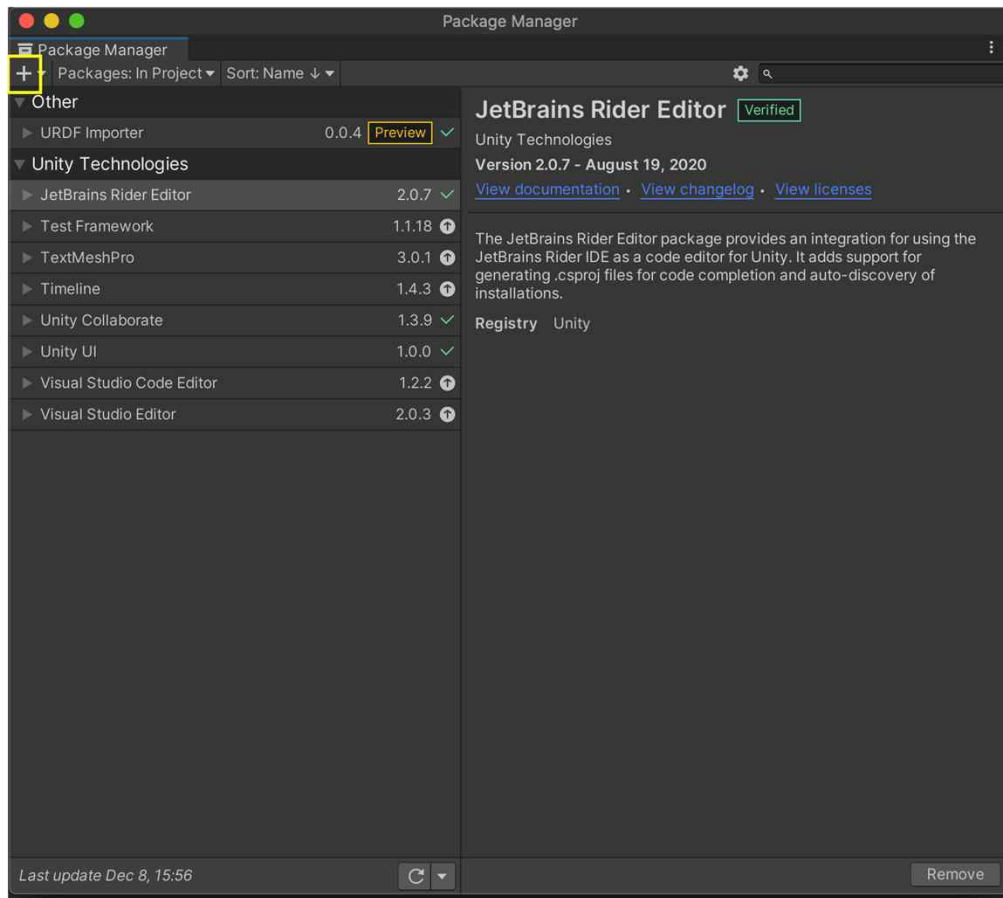
- **Start()**: 게임 시작 시 단 1회 호출 (초기 설정)
- **Update()**: 매 프레임마다 계속 호출 (키보드 입력, 움직임 처리)
- **FixedUpdate()**: 일정한 간격마다 호출 (중력, 충돌 등 물리 계산)
- **OnDestroy()**: 오브젝트가 사라질 때 마지막 1회 호출

03

URDF(Unified Robot Description Format)

- 로봇의 기구학 구조(kinematics)를 XML로 표현
- “어떤 링크들이 어떤 조인트로, 어디에, 어떤 축으로 연결되어 있는가?”를 정의
- ROS, Unity(URDF Importer), Gazebo 등이 같은 URDF를 읽고 → 동일한 로봇 구조를 재구성

Unity - URDF



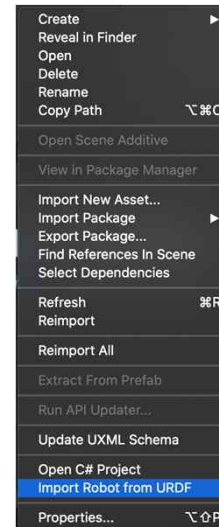
1.상단 Window → Package Manager

2.+ 버튼 → Add Package from Git URL 클릭

3.URL 주소 붙여넣기

<https://github.com/Unity-Technologies/URDF-Importer.git?path=/com.unity.robotics.urdf-importer#v0.5.2>

4.urdf 파일 우클릭 → Import Robot from URDF



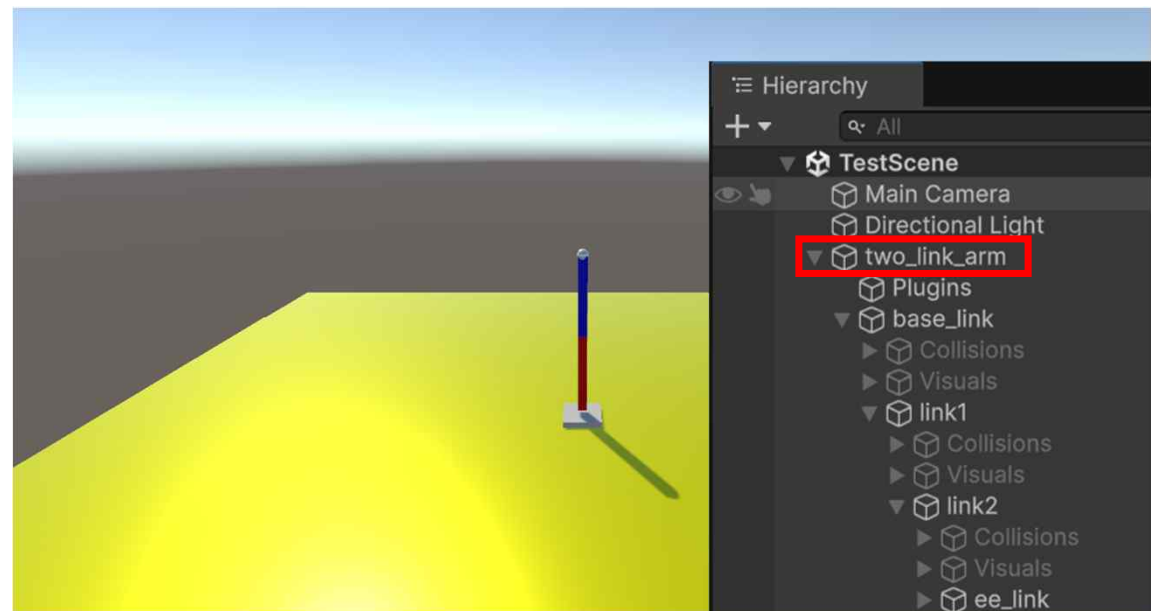
Unity - URDF

URDF 파일 예시

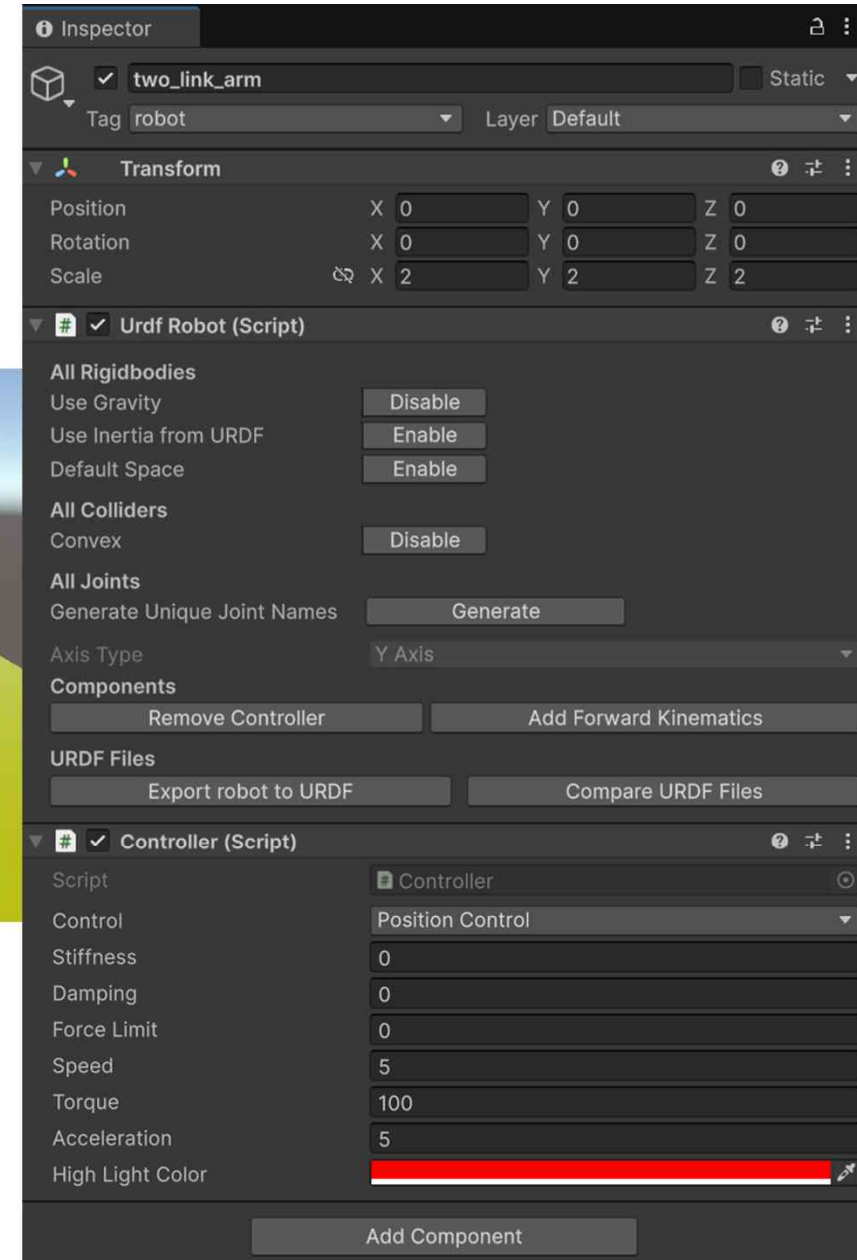
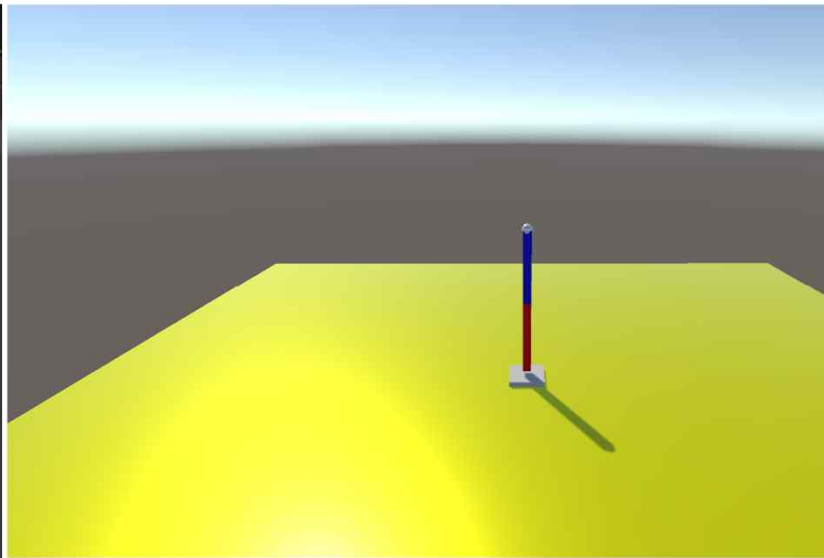
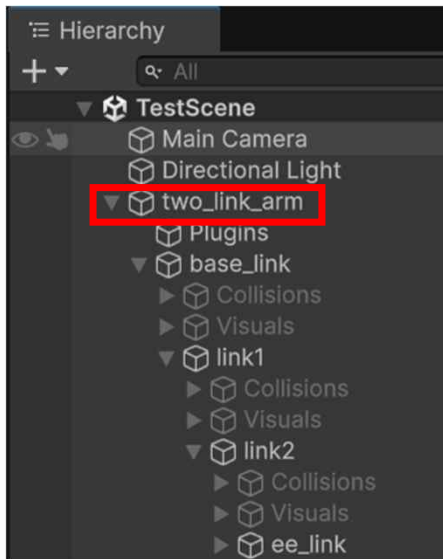
```
<!-- 1번 팔 링크 (길이 0.4m짜리 막대) -->
<link name="link1">
  <visual>
    <!-- 조인트 기준점에서 x축으로 0.2m 떨어진 곳에 중심 -->
    <origin xyz="0.2 0 0" rpy="0 0 0"/>
    <geometry>
      <!-- 길이 0.4m, 두께 0.05m 박스 -->
      <box size="0.4 0.05 0.05"/>
    </geometry>
    <material name="red">
      <color rgba="0.8 0.1 0.1 1.0"/>
    </material>
  </visual>
  <collision>
    <origin xyz="0.2 0 0" rpy="0 0 0"/>
    <geometry>
      <box size="0.4 0.05 0.05"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0.2 0 0" rpy="0 0 0"/>
    <mass value="0.5"/>
    <inertia ixx="0.002" iyy="0.002" izz="0.002"
            ixy="0.0" ixz="0.0" iyz="0.0"/>
  </inertial>
</link>
```



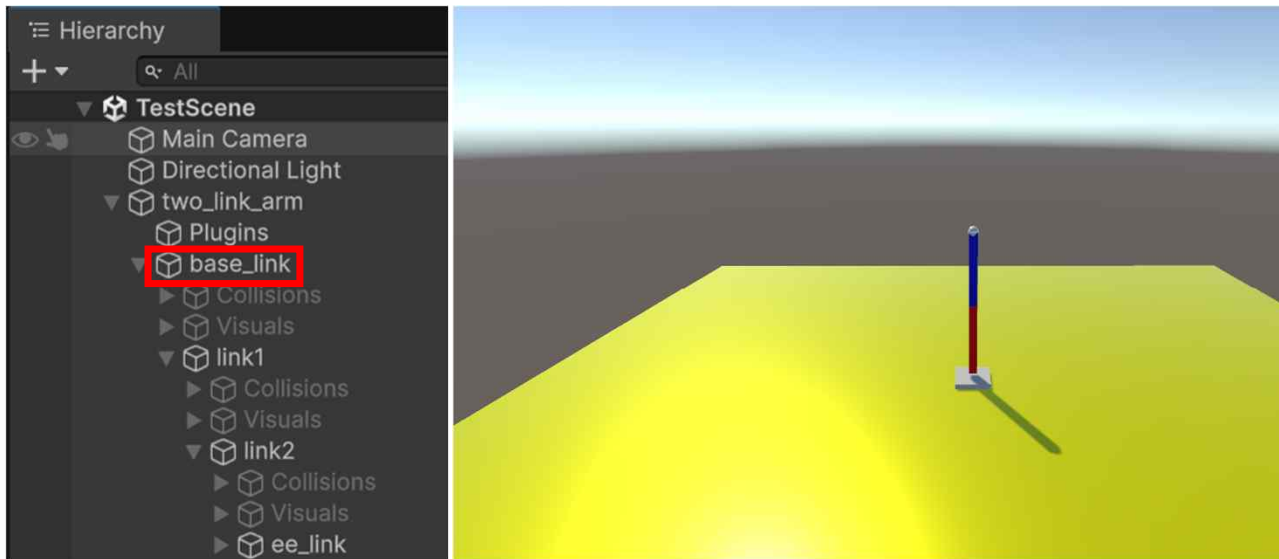
URDF 파일 유니티 임포트 결과



Unity – URDF – 루트 오브젝트 컴포넌트

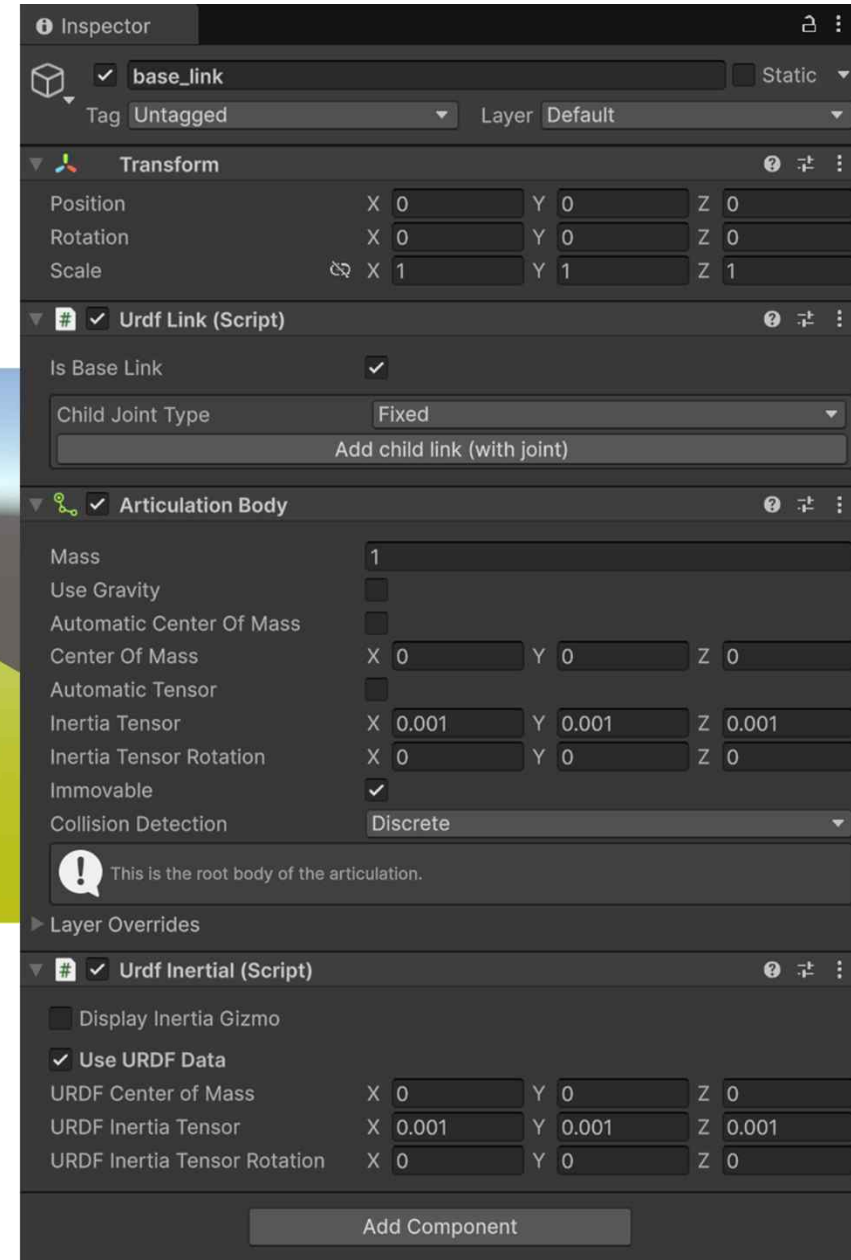


Unity - URDF - 베이스 링크 오브젝트

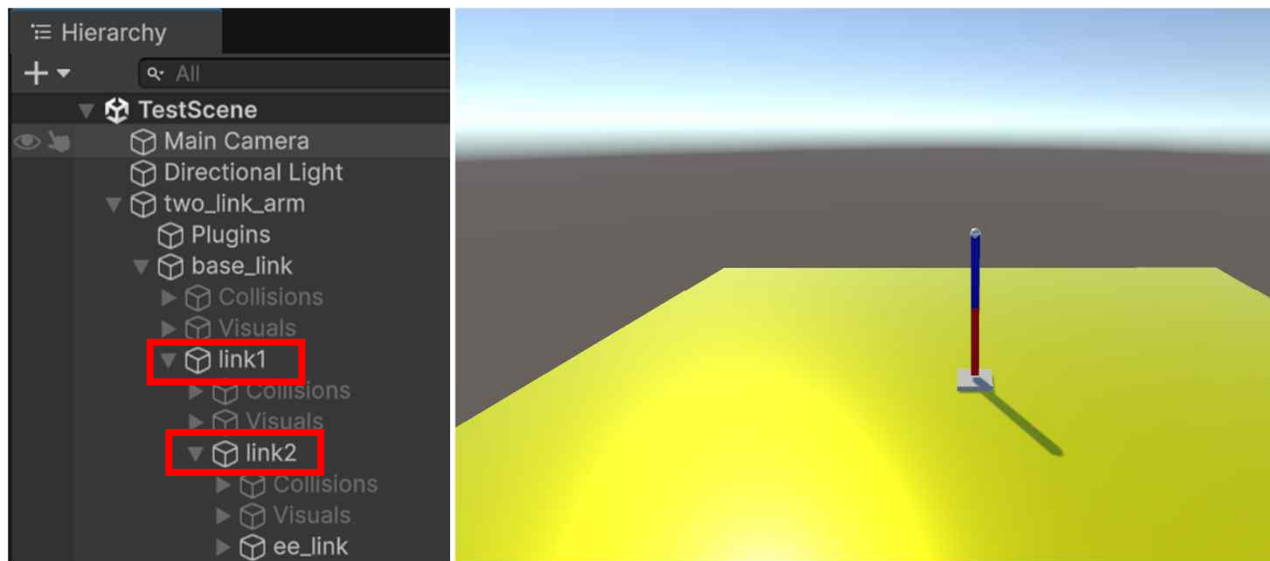


“URDF <link>가 Unity에 오면 UrdfLink + ArticulationBody + UrdfInertial로 쪼개져서 표현”

특히 베이스 링크는 루트 바디라서 Immovable를 켜고 기준 좌표 역할



Unity - URDF - 일반 링크 노드들 1



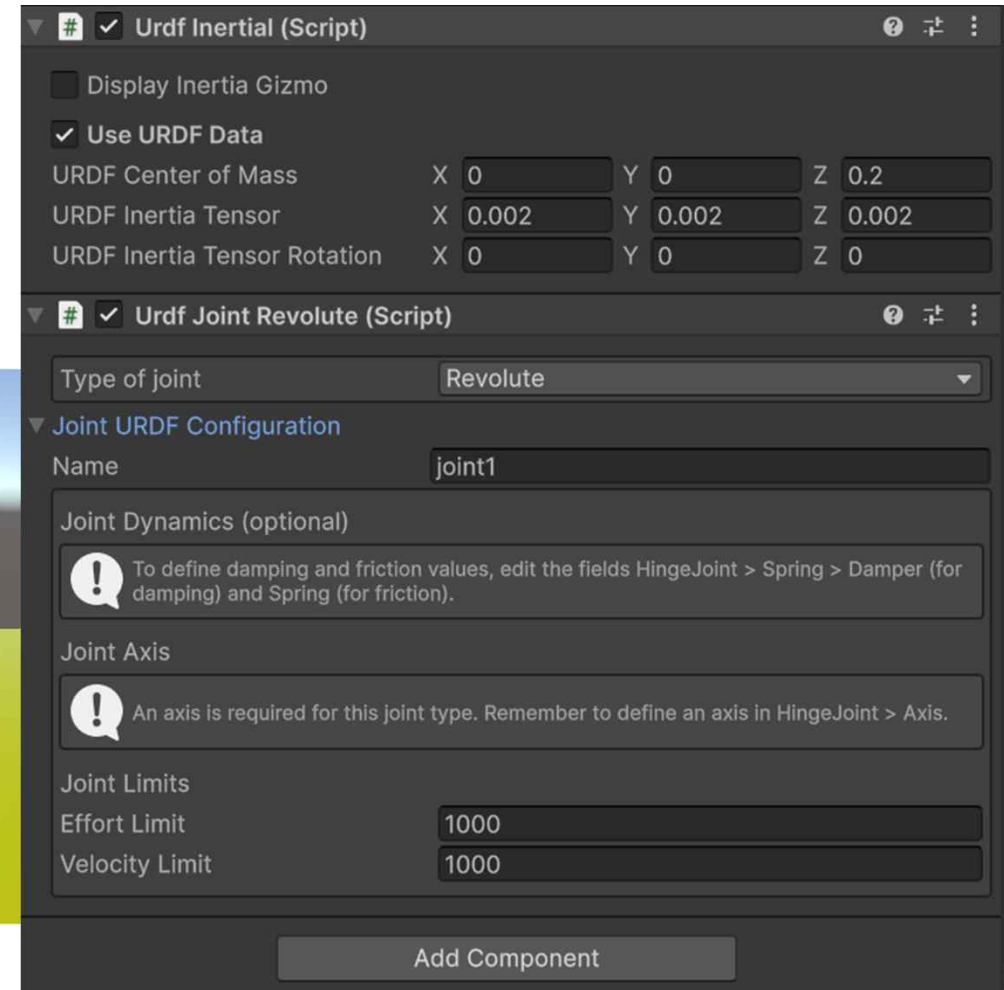
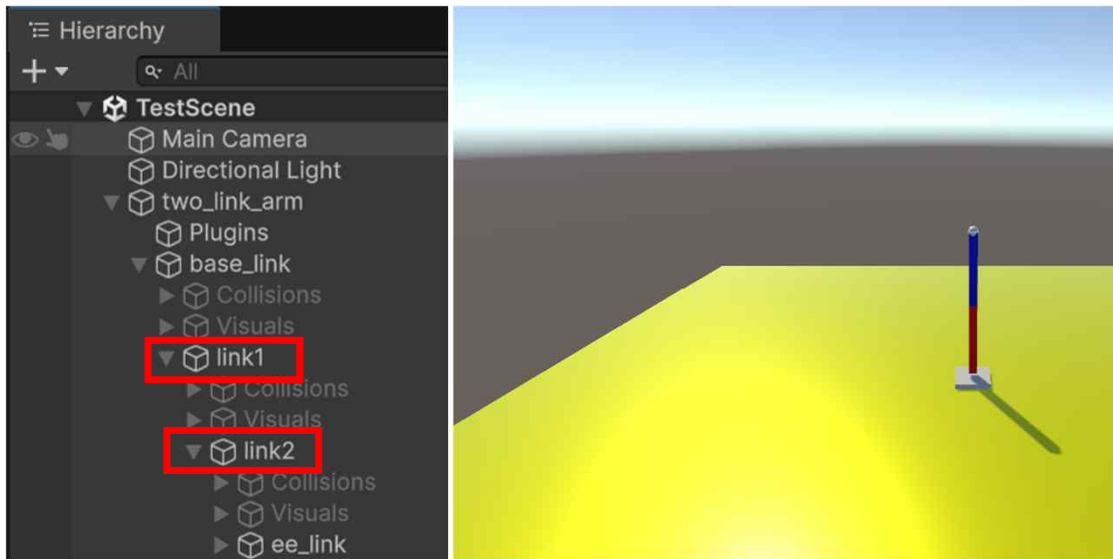
URDF의 <joint> 정보(타입, origin, axis, limit)가 Unity에 오면

→ 자식 링크(link1)의 ArticulationBody 안에 있는

“Articulation Joint Type + Anchor + X Drive” 설정으로 변환



Unity - URDF - 일반 링크 노드들 2

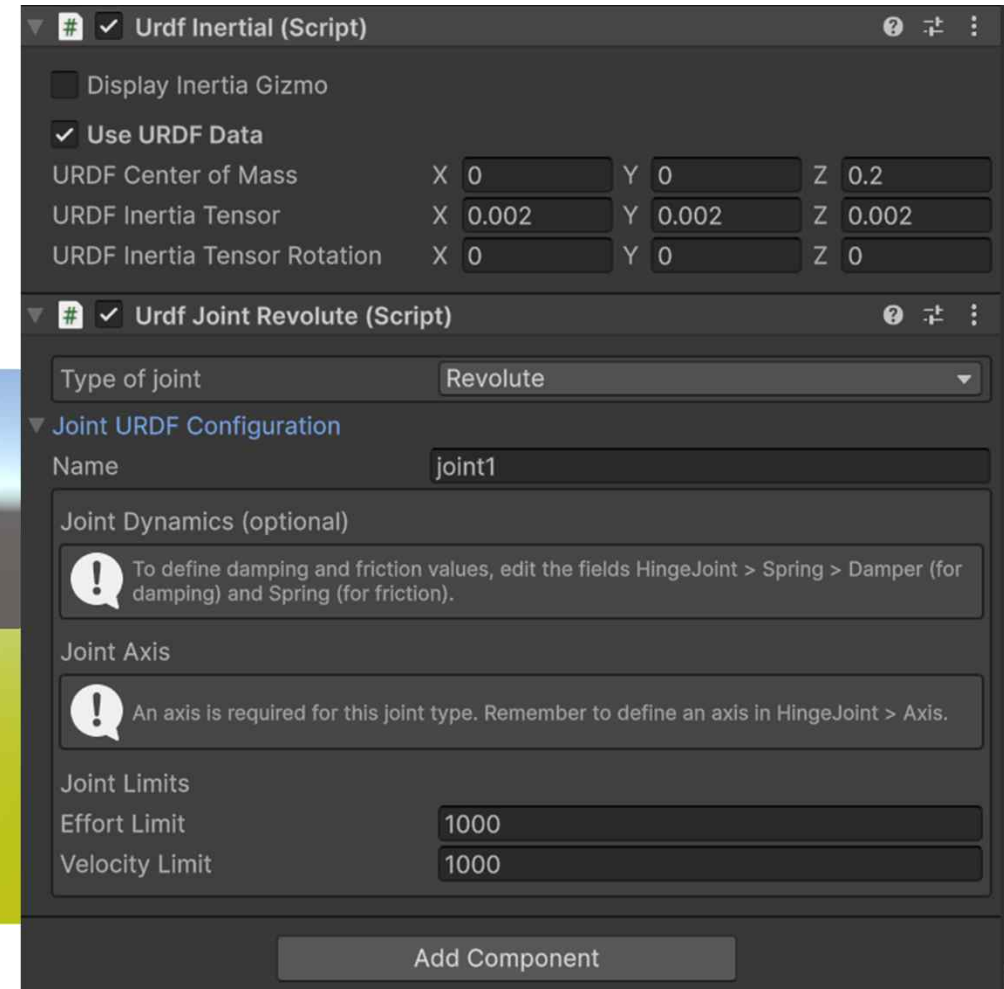
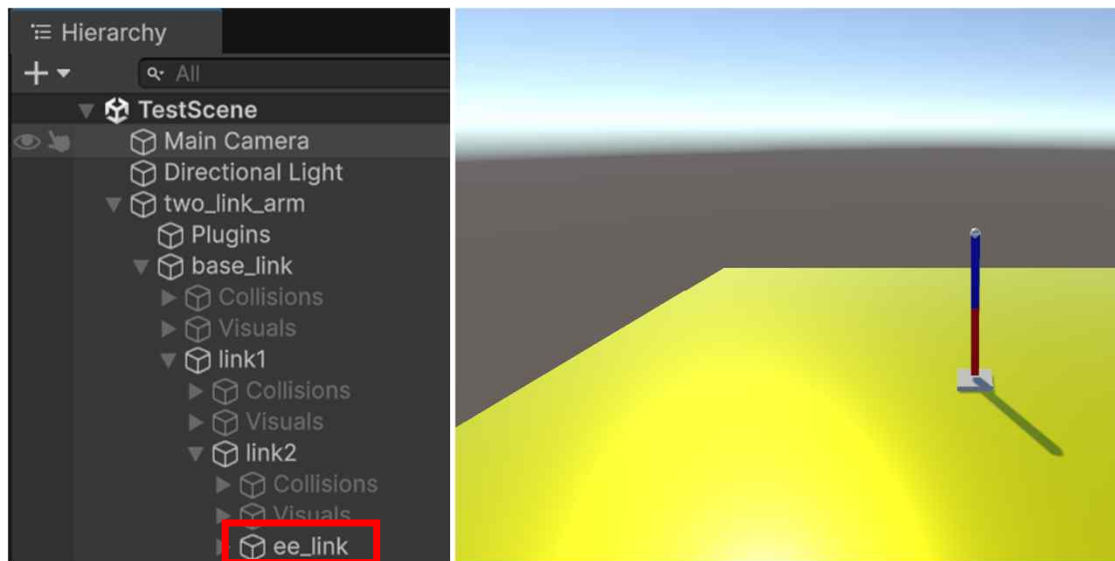


ArticulationBody = 실제 물리 관절

Urdf Joint Revolute = 그 관절의 원본 URDF 설정(이름, 축, 제한, 다이내믹스)을 기억하는 컴포넌트

나중에 Export할 때 이 정보를 모아서 다시 <joint ...> XML로 만들어 준다.

Unity – URDF – ee



ee_link는 팔 끝에 붙은 엔드이펙터 좌표계이고, URDF의 `<joint type="fixed">`가 Unity에 오면,

Articulation Joint Type = Fixed인 링크로 만들어져 상위 링크와 완전히 고정된 하나의 강체처럼 움직임

Unity - URDF - 제어 샘플 코드

```
public class ArmController : MonoBehaviour
{
    public ArticulationBody joint1;
    public ArticulationBody joint2;

    public float stepDeg = 10f;

    float j1Target, j2Target;
    float lower = -90f, upper = 90f;

    Unity 메시지 | 참조 0개
    void Start()
    {
        InitJoint(joint1, ref j1Target);
        InitJoint(joint2, ref j2Target);
    }

    참조 2개
    void InitJoint(ArticulationBody j, ref float target)
    {
        var d = j.xDrive;
        d.stiffness = 1000f;
        d.damping = 100f;
        d.forceLimit = 10000f;
        d.lowerLimit = lower;
        d.upperLimit = upper;
        target = d.target;
        j.xDrive = d;
    }
}
```

```
Unity 메시지 | 참조 0개
void Update()
{
    // joint1 : Up / Down
    if (Input.GetKeyDown(KeyCode.UpArrow)) j1Target += stepDeg;
    if (Input.GetKeyDown(KeyCode.DownArrow)) j1Target -= stepDeg;

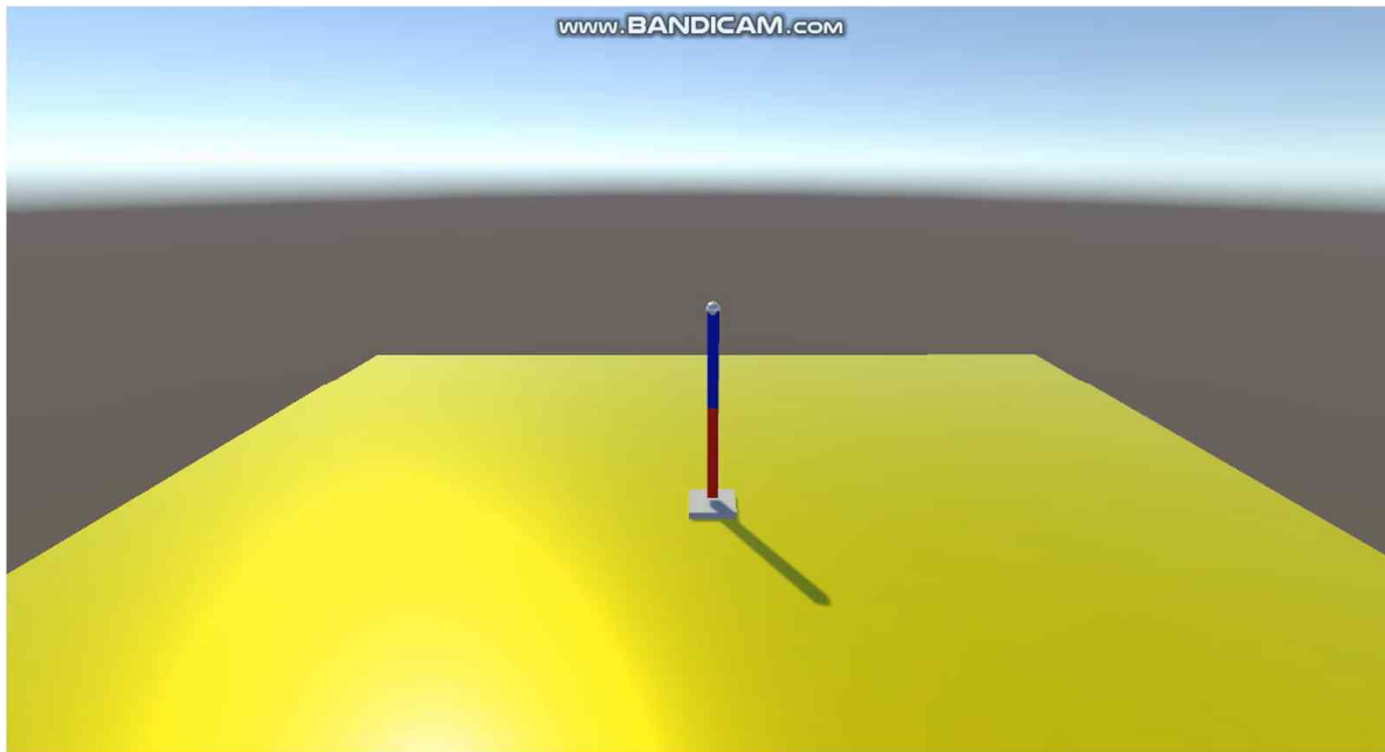
    // joint2 : Left / Right
    if (Input.GetKeyDown(KeyCode.RightArrow)) j2Target += stepDeg;
    if (Input.GetKeyDown(KeyCode.LeftArrow)) j2Target -= stepDeg;

    j1Target = Mathf.Clamp(j1Target, lower, upper);
    j2Target = Mathf.Clamp(j2Target, lower, upper);

    Apply(joint1, j1Target);
    Apply(joint2, j2Target);
}

참조 2개
void Apply(ArticulationBody j, float targetDeg)
{
    var d = j.xDrive;
    d.target = targetDeg;
    j.xDrive = d;
}
```

Unity - URDF - 제어

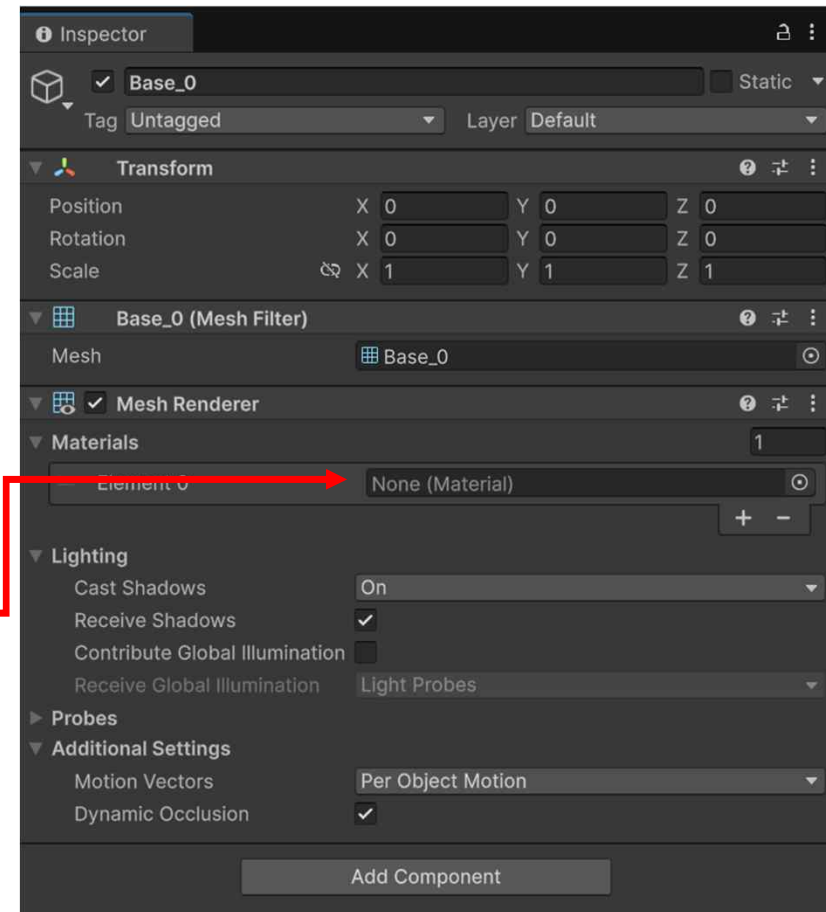
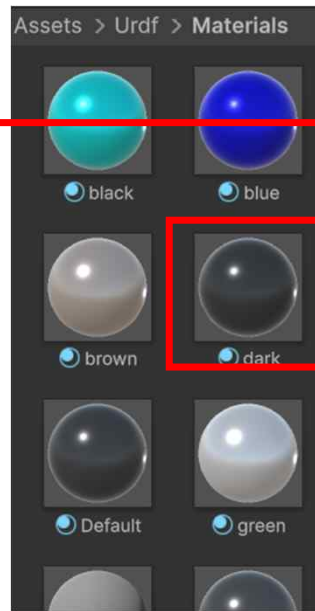
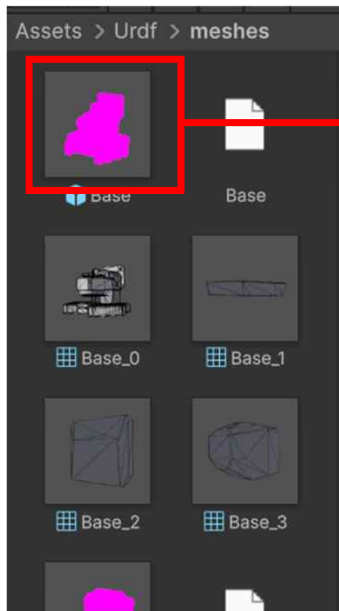


Unity – Lerobot SO100 모델 불러오기

파일 다운로드 : 링크

1. 압축 풀어서 Project > Asset 폴더 하위에 붙여넣기

2. meshes 폴더 안 프리팹들에 머터리얼 채워 넣기



Unity – Lerobot SO100 모델 불러오기 및 실습

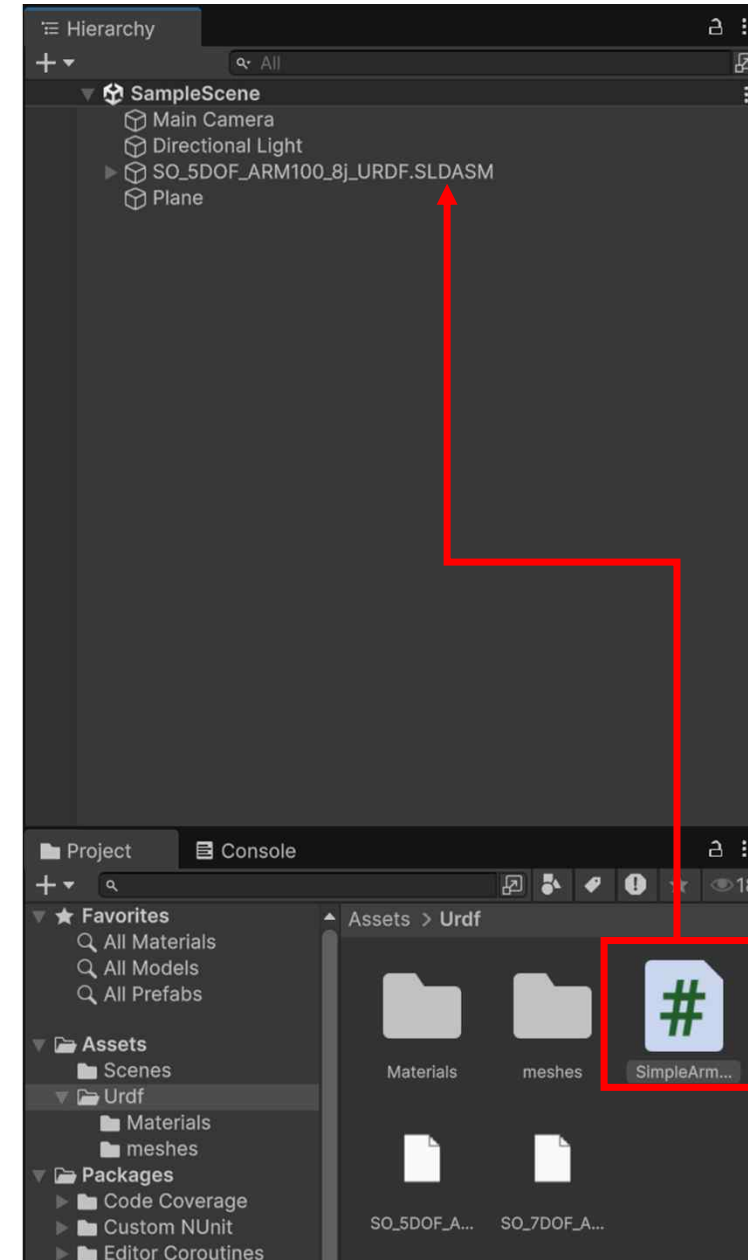
3. SO_5DOF_ARM100_8j_URDF.SLDASM 파일 URDF импорт

4. SOArmKeyboardController.cs 스크립트 생성된 모델에 컴포넌트 추가

(드래그 앤드롭)

5. 키보드로 SO100 로봇 팔 제어 실습

(좌우 화살표로 관절 선택, 위아래 화살표로 제어)



Unity – Lerobot SO100 제어 실습

- 좌우 화살표로 관절 선택
- 위아래 화살표로 모터 제어

